

VCS Concepts

Concepts

VCS is built on three components: **LLT, GAB, and HAD**.

LLT (Low-Latency Transport)

veritas uses a high-performance, low-latency protocol for cluster communications. LLT runs directly on top of the data link provider interface (DLPI) layer over ethernet and has several major junctions:

- sending and receiving heartbeats
- monitoring and transporting network traffic over multiple network links to every active system within the cluster
- load-balancing traffic over multiple links
- maintaining the state of communication
- providing a nonroutable transport mechanism for cluster communications.

Group membership services/Atomic Broadcast (GAB)

GAB provides the following:

- Group Membership Services - GAB maintains the overall cluster membership by the way of its Group Membership Services function. Heartbeats are used to determine if a system is active member, joining or leaving a cluster. GAB determines what the position of a system is in within a cluster.
- Atomic Broadcast - Cluster configuration and status information is distributed dynamically to all system within the cluster using GAB's Atomic Broadcast feature. Atomic Broadcast ensures all active system receive all messages, for every resource and service group in the cluster. Atomic means that all system receives the update, if one fails then the change is rolled back on all systems.

High Availability Daemon (HAD)

The HAD tracks all changes within the cluster configuration and resource status by communicating with GAB. Think of HAD as the manager of the resource agents. A companion daemon called hashadow monitors HAD and if HAD fails hashadow attempts to restart it. Like wise if hashadow daemon dies HAD will restart it. HAD

maintains the cluster state information. HAD uses the main.cf file to build the cluster information in memory and is also responsible for updating the configuration in memory.

VCS architecture

So putting the above altogether we get:

- Agents monitor resources on each system and provide status to HAD on the local system
- HAD on each system send status information to GAB
- GAB broadcasts configuration information to all cluster members
- LLT transports all cluster communications to all cluster nodes
- HAD on each node takes corrective action, such as failover, when necessary

Service Groups

There are three types of service groups:

- **Failover** - The service group runs on one system at any one time.
- **Parallel** - The service group can run simultaneously on more than one system at any time.
- **Hybrid** - A hybrid service group is a combination of a failover service group and a parallel service group used in VCS 4.0 replicated data clusters, which are based on Veritas Volume Replicator.

When a service group appears to be suspended while being brought online you can flush the service group to enable corrective action. Flushing a service group stops VCS from attempting to bring resources online or take them offline and clears any internal wait states.

Resources

Resources are objects that related to hardware and software, VCS controls these resources through these actions:

- Bringing resource online (starting)
- Taking resource offline (stopping)
- Monitoring a resource (probing)

When you link a parent resource to a child resource, the dependency becomes a

component of the service group configuration. You can view the dependencies at the bottom of the main.cf file.

Proxy Resource

A proxy resource allows multiple service groups to monitor the same network interface. This reduces the network traffic that would result from having multiple NIC resources in different service groups monitoring the same interface.

Example for Proxy Resource:

```
Proxy PreProd_proxy (  
  
    Critical = 0  
    TargetResName = PreProd_MultiNICB  
)
```

Phantom Resource

The phantom resource is used to report the actual status of a service group that consists of only persistent resources. A service group shows an online status only when all of its nonpersistent resources are online. Therefore, if a service group has only persistent resources (network interface), VCS considers the group offline, even if the persistent resources are running properly. By adding a phantom resource, the status of the service group is shown as online.

Example for Phantom:

```
Phantom Phantom_NIC (  
  
)
```

Configuration

VCS configuration is fairly simple. The three configurations to worry about are LLT, GAB, and VCS resources.

LLT

LLT configuration requires two files: /etc/llttab and /etc/llthosts. llttab contains information on node-id, cluster membership, and heartbeat links. It should look like this:
llttab -- low-latency transport configuration file

this sets our node ID, must be unique in cluster

set-node 0

```
# set the heartbeat links
link hme1 /dev/hme:1 - ether - -
# link-lowpri is for public networks
link-lowpri hme0 /dev/hme:0 - ether - -

# set cluster number, must be unique
set-cluster 0
```

start

The "link" directive should only be used for private links. "link-lowpri" is better suited to public networks used for heartbeats, as it uses less bandwidth. VCS requires at least two heartbeat signals (although one of these can be a communication disk) to function without complaints.

The "set-cluster" directive tells LLT which cluster to listen to. The lltab needs to end in "start" to tell LLT to actually run.

The second file is /etc/llthosts. This file is just like /etc/hosts, except instead of IP->hostnames, it does ll node numbers (as set in set-node). You need this file for VCS to start. It should look like this:

```
0    daldev05
1    daldev06
```

GAB

GAB requires only one configuration file, /etc/gabtab. This file lists the number of nodes in the cluster and also, if there are any communication disks in the system, configuration for them. Ex:

```
/sbin/gabconfig -c -n2
```

tells GAB to start GAB with 2 hosts in the cluster. To specify VCS communication disks:

```
/sbin/gabdisk -a /dev/dsk/cXtXdXs2 -s 16 -p a
/sbin/gabdisk -a /dev/dsk/cXtXdXs2 -s 144 -p h
/sbin/gabdisk -a /dev/dsk/cYtYdYs2 -s 16 -p a
/sbin/gabdisk -a /dev/dsk/cYtYdYs2 -s 144 -p h
```

-a specifies the disk, -s specifies the start block for each communication region, and -p specifies the port to use, "a" being the GAB seed port and "h" the VCS port. The ports are the same as the network ports used by LLT and GAB, but are simulated on a disk.

VCS

The VCS configuration file(s) are in [**/etc/VRTSvcs/conf/config**](#). The two most

important files are main.cf and types.cf. I like to set \$VCSCONF to that directory to make my life easier. main.cf contains the actual VCS configuration for Clusters, Groups, and Resources, while types.cf contains C-like prototypes for each possible Resource. The VCS configuration is very similar to the C language, but all you are doing is defining variables. Comments are "//" (if you try to use #'s, you'll be unhappy with the result), and you can use "include" statements if you want to break up your configuration to make it more readable. One file you must include is types.cf.

In main.cf, you need to specify a Cluster definition:

```
cluster iMS (
```

You can specify variables within this cluster definition, but for the most part, the defaults are acceptable. Cluster variables include maximum number of groups per cluster, link monitoring, log size, maximum number of resources, maximum number of types, and a list of user names for the GUI that you will never use and shouldn't install.

You then need to specify the systems in the cluster:

```
system daldev05 (
```

```
system daldev06 (
```

These systems must be in /etc/llhosts for VCS to start.

You can also specify SNMP settings for VCS:

```
snmp vcs (
```

```
    Enabled = 1
```

```
    IPAddr = 0.0.0.0
```

```
    TrapList = { 1 = "A new system has joined the VCS Cluster",
```

```
                2 = "An existing system has changed its state",
```

```
                3 = "A service group has changed its state",
```

```
                4 = "One or more heartbeat links has gone down",
```

```
                5 = "An HA service has done a manual restart",
```

```
                6 = "An HA service has been manually idled",
```

```
                7 = "An HA service has been successfully started" }
```

```
)
```

IPAddr is the IP address of the trap listener. Enabled defaults to 0, so you need to include this if you want VCS to send traps. You can also specify a list of numerical traps; listed above are the VCS default traps.

Each cluster can have multiple Service Group definitions. The most basic Service Group looks like this:

```
group iMS5a (
```

```
    SystemList = { daldev05, daldev06 }
```

```
    AutoStartList = { daldev05 }
```

```
)
```

You can also set the following variables (not a complete list):

- FailOverPolicy - you can set which policy is used to determine which system to fail

over to, choose from Priority (numerically based on node-id), Load (system with the lowest system load gets failover), or RoundRobin (system with the least number of active services is chosen).

- ManualOps - whether VCS allows manual (CLI) operation on this Group
- Parallel - indicates if the service group is parallel or failover

Inside each Service Group you need to define Resources. These are the nuts and bolts of VCS. A full description of the bundled Resources can be found in the Install Guide and a full description of the configuration language can be found in the User's Guide.

Here are a couple of Resource examples:

```
NIC networka (  
    Device = hme0  
    NetworkType = ether  
)
```

```
IP logical_IPa (  
    Device = hme0  
    Address = "10.10.30.156"  
)
```

The first line begins with a Resource type (e.g. NIC or IP) and then a globally unique name for that particular resource. Inside the paren block, you can set the variables for each resource.

Once you have set up resources, you need to build a resource dependancy tree for the group. The syntax is "child_resource requires parent_resource." A dependancy tree for the above resources would look like this:

```
logical_IPa requires networka
```

The dependancy tree tells VCS which resources need to be started before other resources can be activated. In this case, VCS knows that the NIC hme0 has to be working before resource logical_IPa can be started. This works well with things like volumes and volumegroups; without a dependancy tree, VCS could try to mount a volume before importing the volume group. VCS deactivates all VCS controlled resources when it shuts down, so all virtual interfaces (resource type IP) are unplumbed and volumes are unmounted/exported at VCS shutdown.

Once the configuration is build, you can verify it by running /opt/VRTSvcs/bin/hacf -verify and then you can start VCS by running /opt/VRTSvcs/bin/hastart.

Commands and Tasks

Here are some important commands in VCS. They are in **/opt/VRTSvcs/bin** unless otherwise noted. It's a good idea to set your PATH to include that directory.

Manpages for these commands are all installed in /opt/VRTS/man.

- **hastart** starts VCS using the current seeded configuration.

- **hastop** stops VCS. -all stops it on all VCS nodes in the cluster, -force keeps the service groups up but stops VCS, and -local stop VCS on the current node, and -sys systemname stop VCS on a remote system.
- **hastatus** shows VCS status for all nodes, groups, and resources. It waits for new VCS status, so it runs forever unless you run it with the -summary option.
- **/sbin/lltstat** shows network statistics (for only the local host) much like netstat -s. Using the -nvv option shows detailed information on all hosts on the network segment, even if they aren't members of the cluster.
- **/sbin/gabconfig** sets the GAB configuration just like in /etc/gabtab. /sbin/gabconfig -a show current GAB port status. Output should look like this:
- daldev05 # **/sbin/gabconfig -a**

GAB Port Memberships

=====

```
Port a gen f6c90005 membership 01
```

```
Port h gen 3aab0005 membership 01
```

The last digits in each line are the node IDs of the cluster members. Any mention of "jeopardy" ports means there's a problem with that node in the cluster.

- **haclus** displays information about the VCS cluster. It's not particularly useful because there are other, more detailed tools you can use:
- **hasys** controls information about VCS systems. hasys -display shows each host in the cluster and it's current status. You can also set this to add, delete, or modify existing systems in the cluster.
- **hagrp** controls Service Groups. It can offline, online (or swing) groups from host to host. This is one of the most useful VCS tools.
- **hares** controls Resources. This is the finest granular tool for VCS, as it can add, remove, or modify individual resources and resource attributes.

Here are some useful things you can do with VCS:

Activate VCS: run "**hastart**" on one system. All members of the cluster will use the seeded configuration. All the resources come up.

Swing a whole Group administratively:

Assuming the system you're running GroupA on is sysa, and you want to swing it to sysb

hagrp -switch GroupA -to sysb

Turn off a particular resource (say, ResourceA on sysa):

hares -offline ResourceA -sys sysa

In a failover Group, you can only online the resource on system on which the group is online, so if ResourceA is a member of GroupA, you can only bring ResourceA online on the system that is running GroupA. To online a resource:

hares -online ResourceA -sys sysa

If you get a fault on any resource or group, you need to clear the Fault on a system

before you can bring that resource/group up on it. To clear faults:

hagrp -clear GroupA

hares -clear ResourceA

Caveats

Here are some tricks for VCS:

VCS likes to have complete control of all its resources. It brings up all its own virtual interfaces, so don't bother to do that in your init scripts. VCS also likes to have complete control of all the Veritas volumes and groups, so you shouldn't mount them at boot. VCS will fail to mount a volume unless it is responsible for importing the Volume Group; if you import the VG and then start VCS, it will fail after about 5 minutes and drop the volume without cleaning the FS. So make sure all VCS-controlled VG's are exported before starting VCS.

Resource and Group names have no scope in VCS, so each must be a unique identifier or VCS will fail to load your new configuration. There is no equivalent to perl's my or local. VCS is also very case sensitive, so all Types, Groups, Resources, and Systems must be the same every time. To make matters worse, most of the VCS bundled types use random capitalization to try to fool you.

Make sure to create your Resource Dependency Tree before you start VCS or it will create a problem to your whole cluster.

The default time-out for LLT/GAB communication is 15 seconds. If VCS detects a system is down on all communications channels for 15 seconds, it fails all of that system's resource groups over to a new system.

If you use Veritas VM, VCS can't manage volumes in rootdg, so what I do is encapsulate the root disk into rootdg and create new volume in their own VCS managed VG. Don't put VCS and non-VCS volumes in the same VG.

Don't let VCS manage non-virtual interfaces. I did this in testing, and if you fail a real interface, VCS will unplumb it, fail it over to a virtual on the fail-over system. Then when you try to swing it back, it will fail.

Notes on how the configuration is loaded

Because VCS doesn't have any determination of primary/slave for the cluster, VCS needs to determine who has the valid configuration for the cluster. As far as I can tell, this is how it works: When VCS starts, GAB waits a predetermined timeout for the number of systems in /etc/gabtab to join the cluster. At this point, all the systems in the cluster compare local configurations, and the system with the newest config tries to load it. If it's invalid, it pulls down the second newest valid config. If it is valid, all the systems in VCS load that config.